

AVL

AVL Tree

- AVL tree is a self-balancing binary search tree in which each node maintains extra information called a balance factor whose value is either -1, 0 or +1.
- AVL tree got its name after its inventor Georgy Adelson-Velsky and Landis.

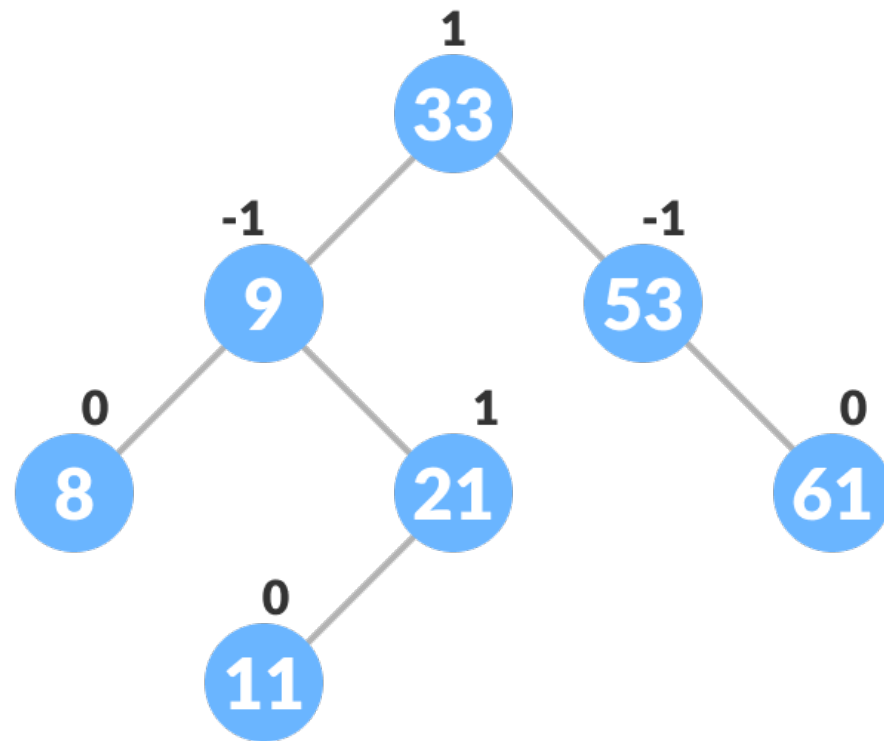
Balance Factor

- The balance factor of a node in an AVL tree is the difference between the height of the left subtree and that of the right subtree of that node.

Balance Factor = (Height of Left Subtree - Height of Right Subtree) or (Height of Right Subtree - Height of Left Subtree)

- The self-balancing property of an AVL tree is maintained by the balance factor.
- The value of the balance factor should always be -1, 0, or +1.

Example

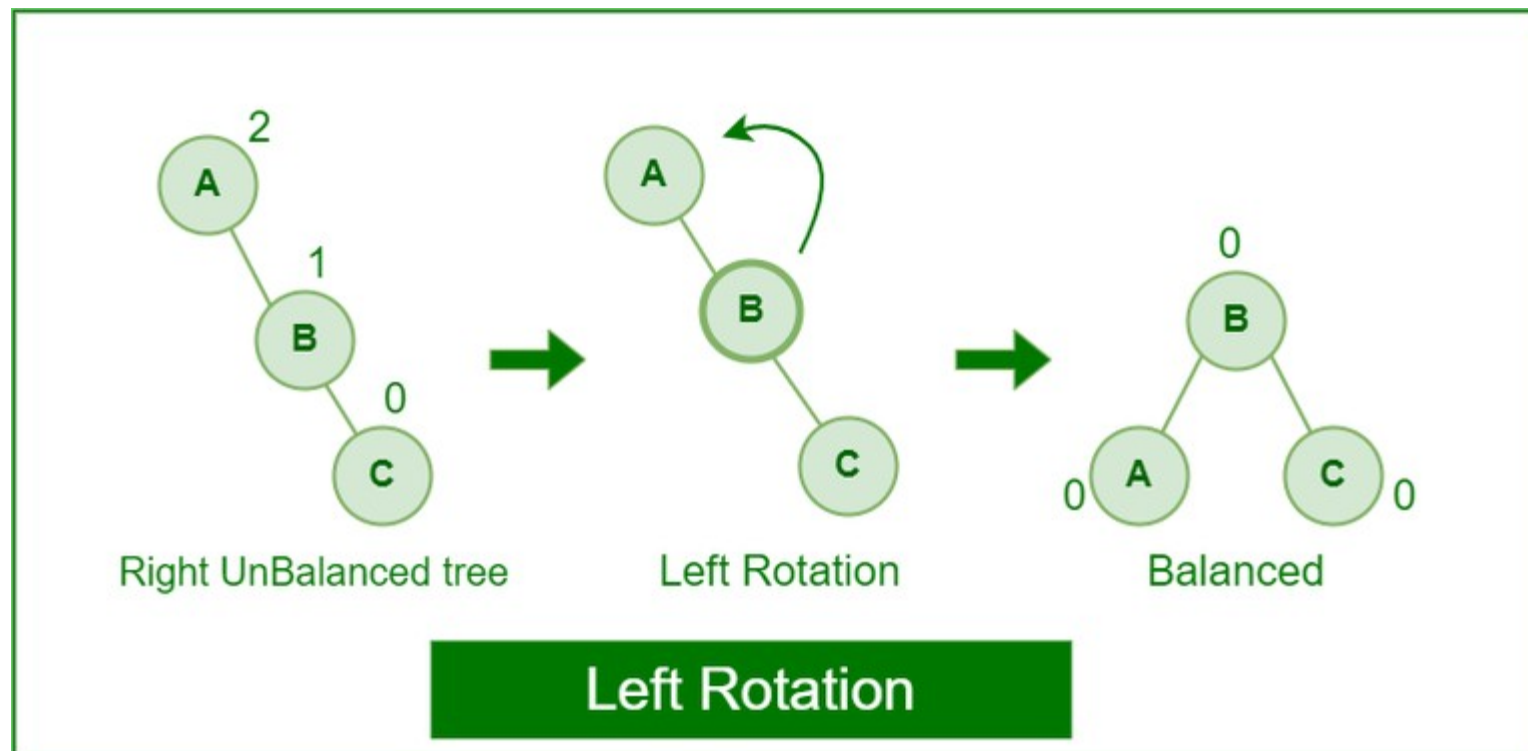


Rotation in AVL

- In rotation operation, the positions of the nodes of a subtree are interchanged.
- There are four types of rotations:
 - Left Rotation
 - Right Rotation
 - Left-Right Rotation
 - Right-Left Rotation

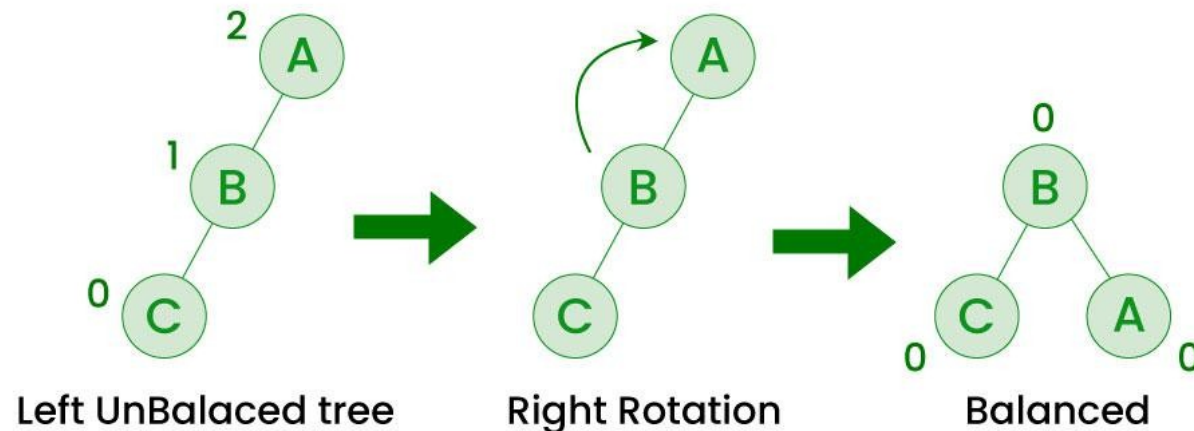
Left Rotation

- When a node is added to the right subtree of the right subtree if the tree gets out of balance, we do a single left rotation.



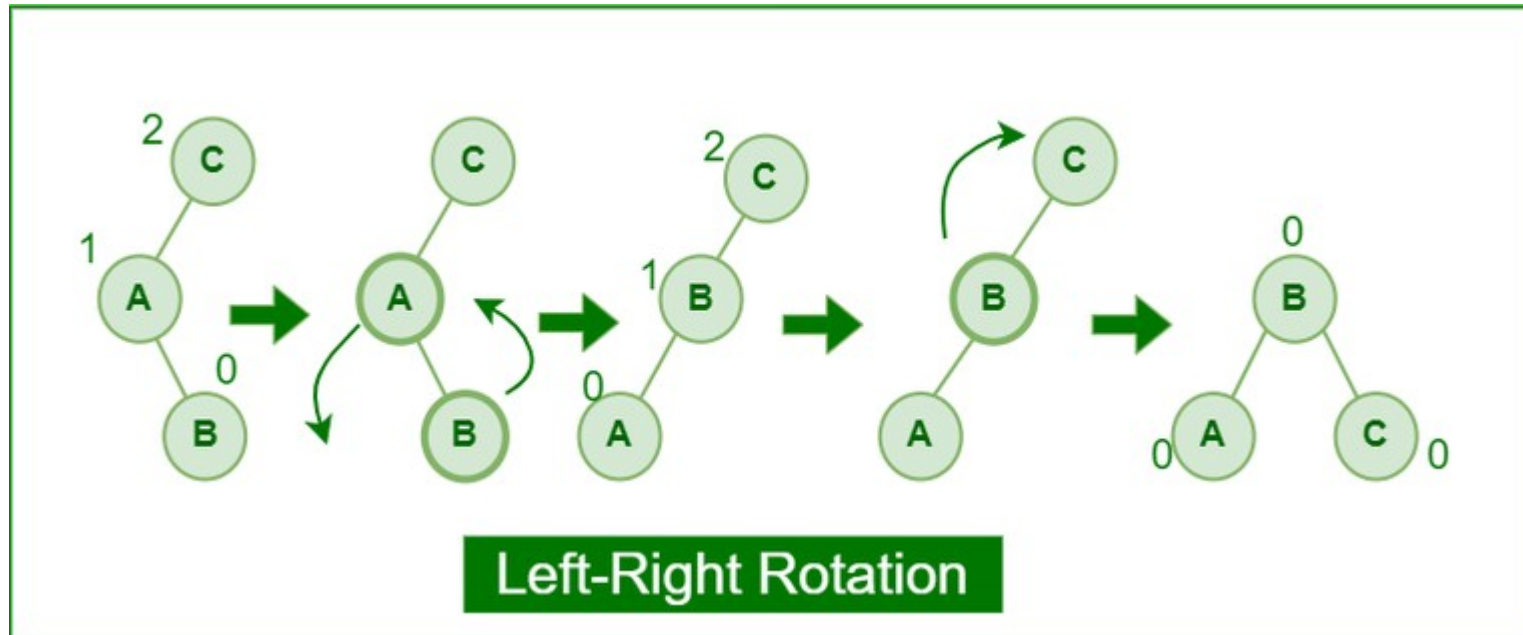
Right Rotation

- If a node is added to the left subtree of the left subtree, the AVL tree may get out of balance, we do a single right rotation



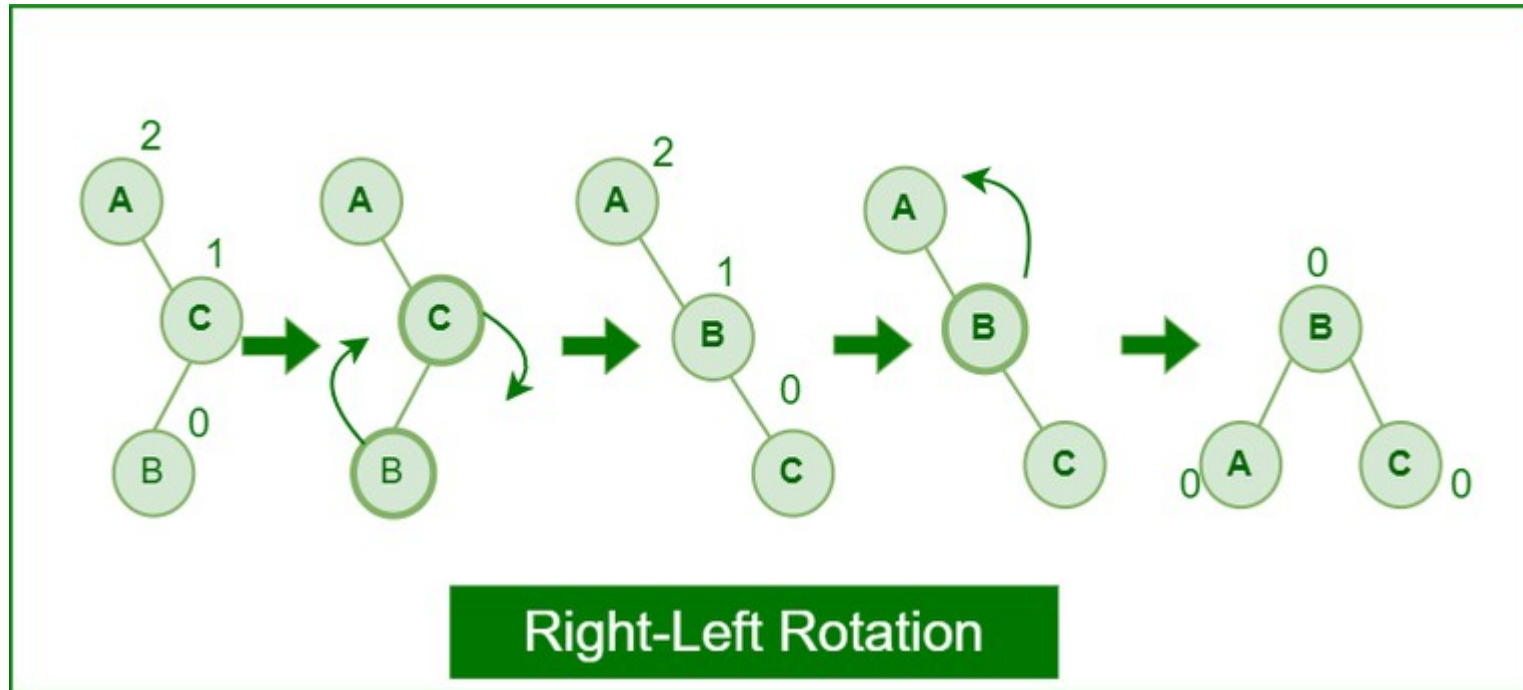
Left-Right Rotation

- A left-right rotation is a combination in which first left rotation takes place after the right rotation executes.

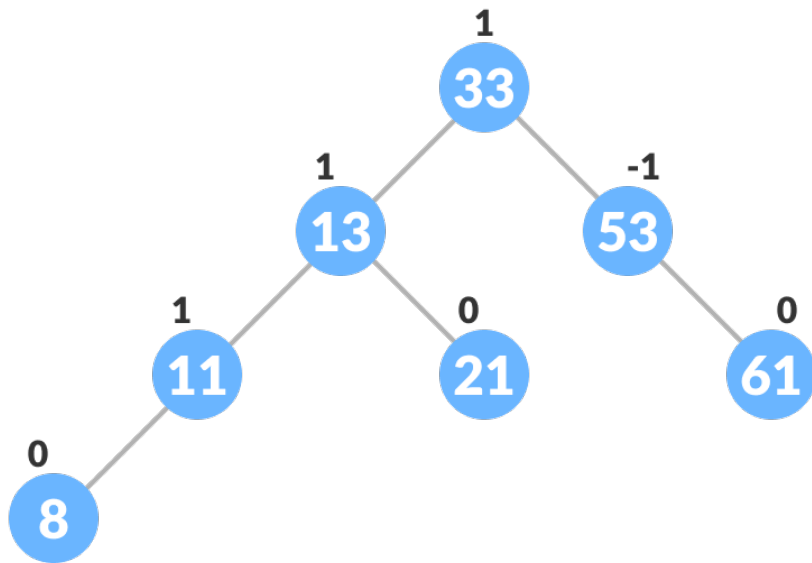


Right-Left Rotation

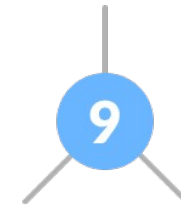
- A right-left rotation is a combination in which first right rotation takes place after the left rotation executes



Insertion



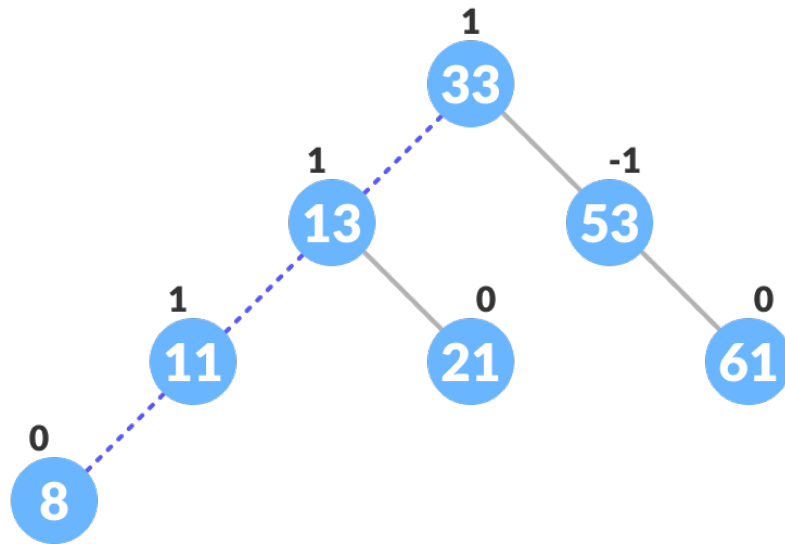
Initial tree for insertion



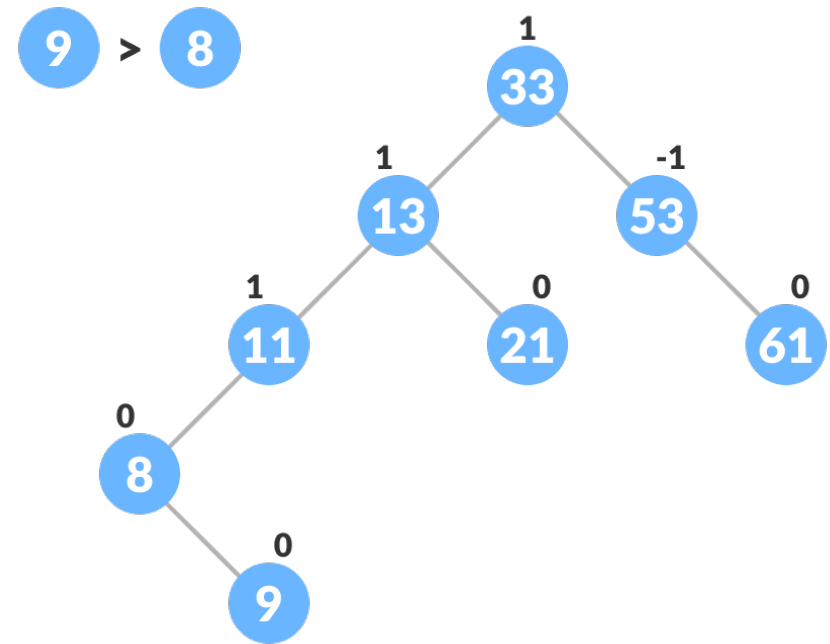
node to be inserted

Insertion

9 < 33
9 < 13
9 < 11

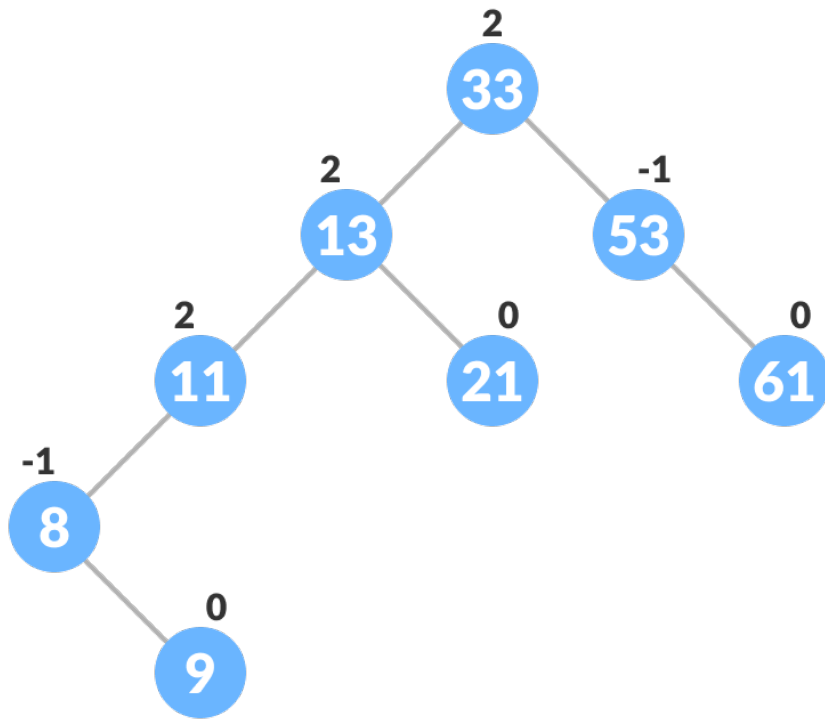


Finding the location to insert

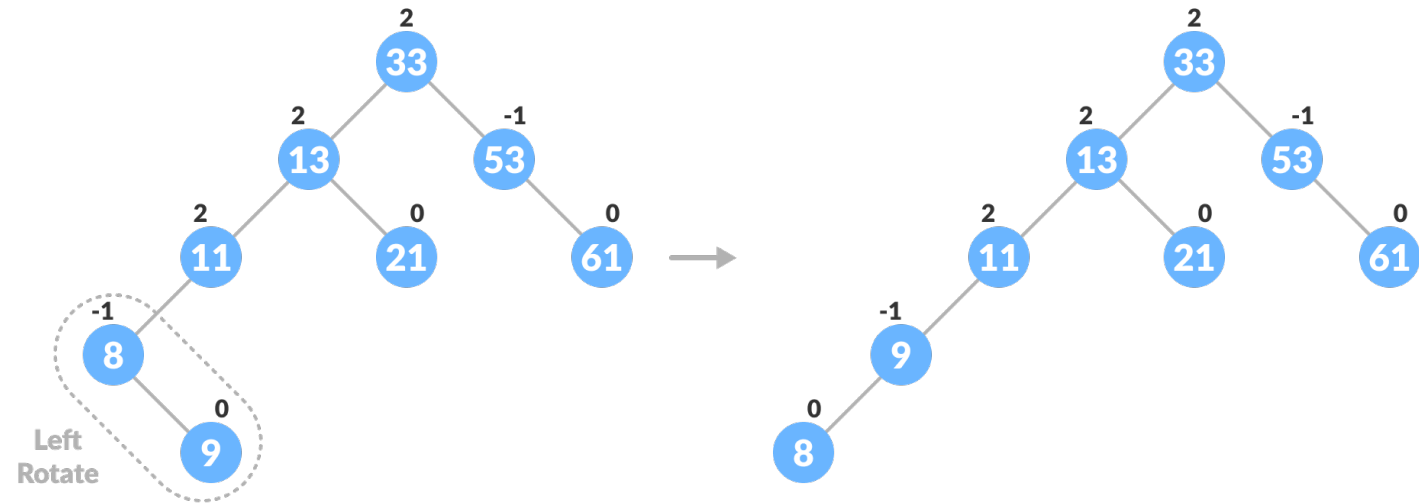


Inserting the new node

Insertion

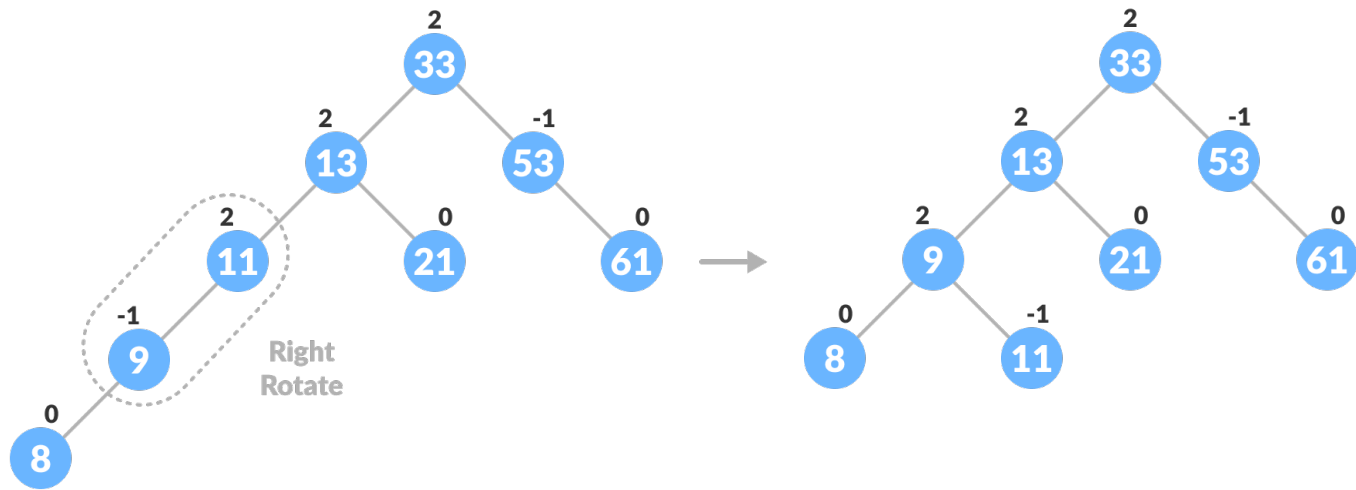


Updating the balance factor after insertion

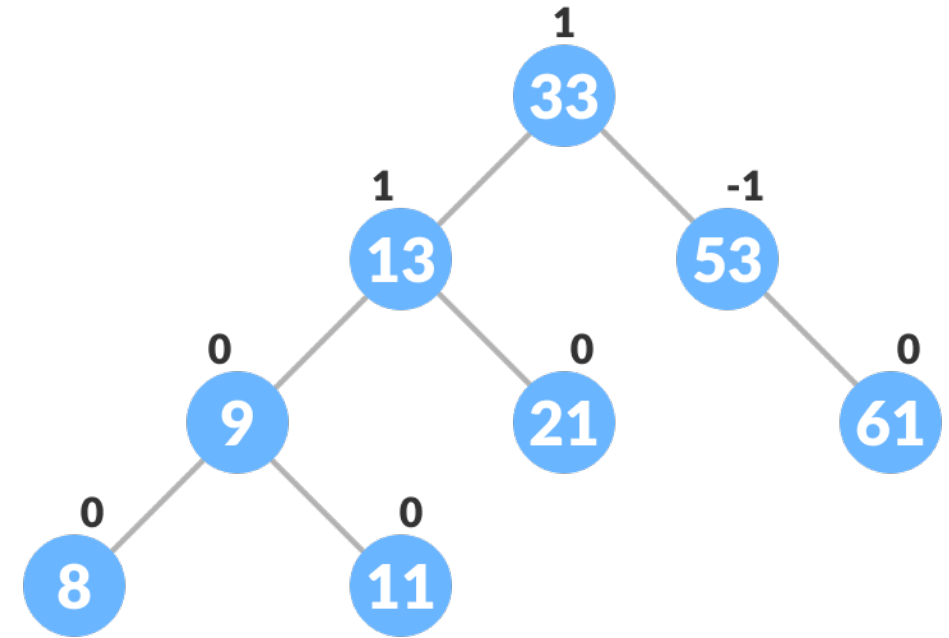


Balancing the tree with rotation

Insertion

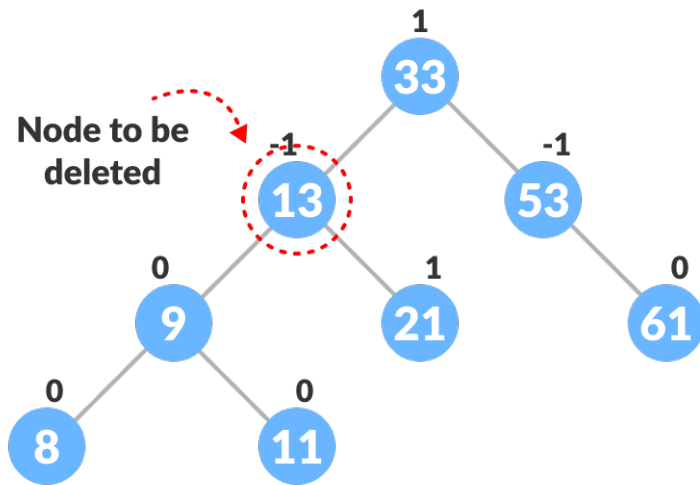


Balancing the tree with rotation



Final balanced tree

Deletion

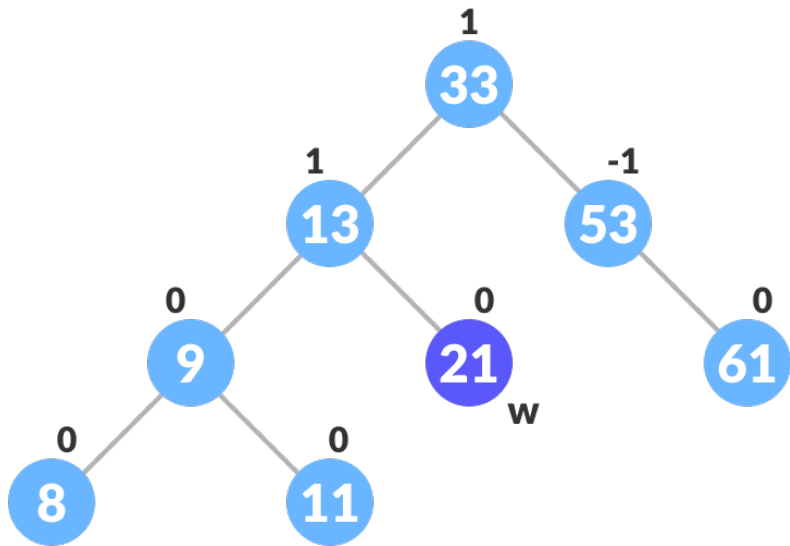


Locating the node to be deleted

There are three cases for deleting a node:

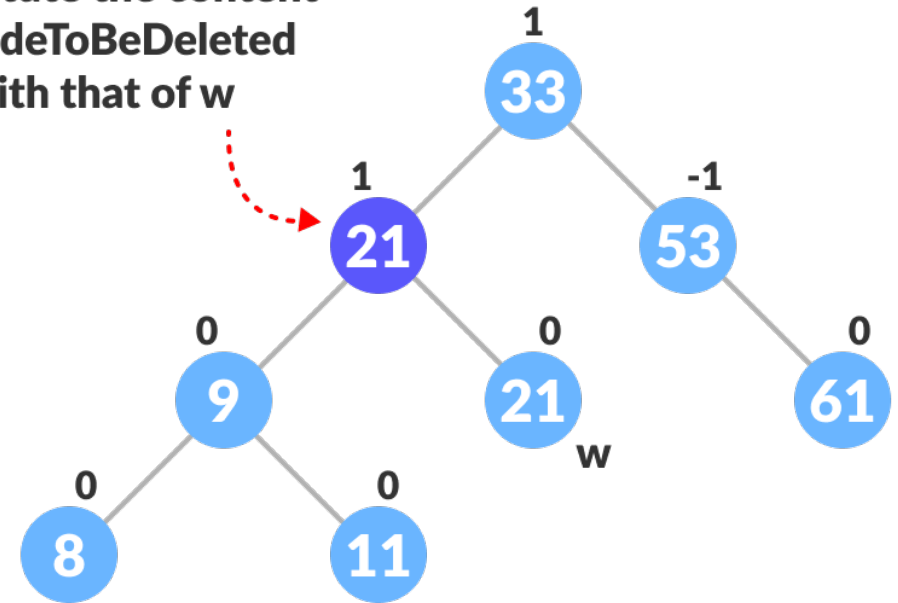
- If **nodeToBeDeleted** is the **leaf** node (ie. does not have any child), then remove **nodeToBeDeleted**.
- If **nodeToBeDeleted** has **one** child, then substitute the contents of **nodeToBeDeleted** with that of the child. Remove the child.
- If **nodeToBeDeleted** has **two** children, find the in-order successor **w** of **nodeToBeDeleted** (ie. node with a minimum value of key in the right subtree).

Deletion



Finding the successor

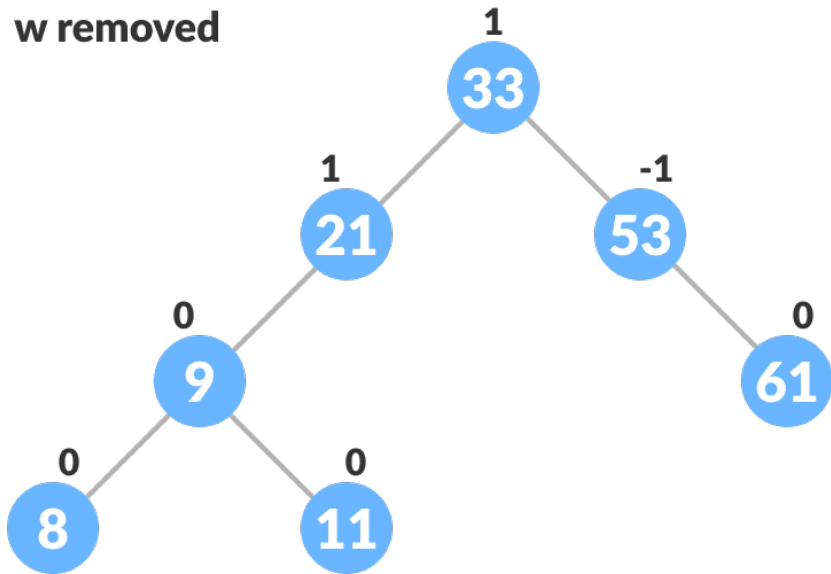
Substitute the content
of nodeToBeDeleted
with that of w



Substitute the node to be deleted

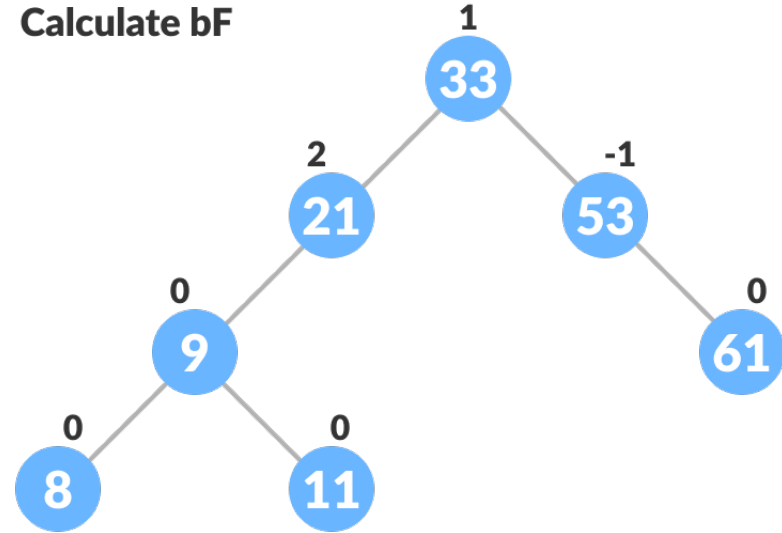
Deletion

w removed



Remove w

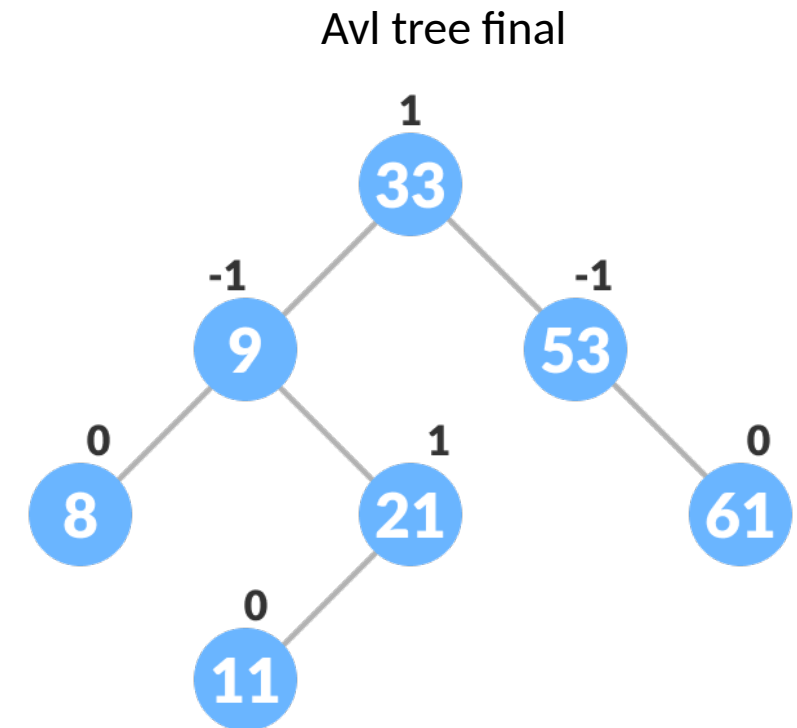
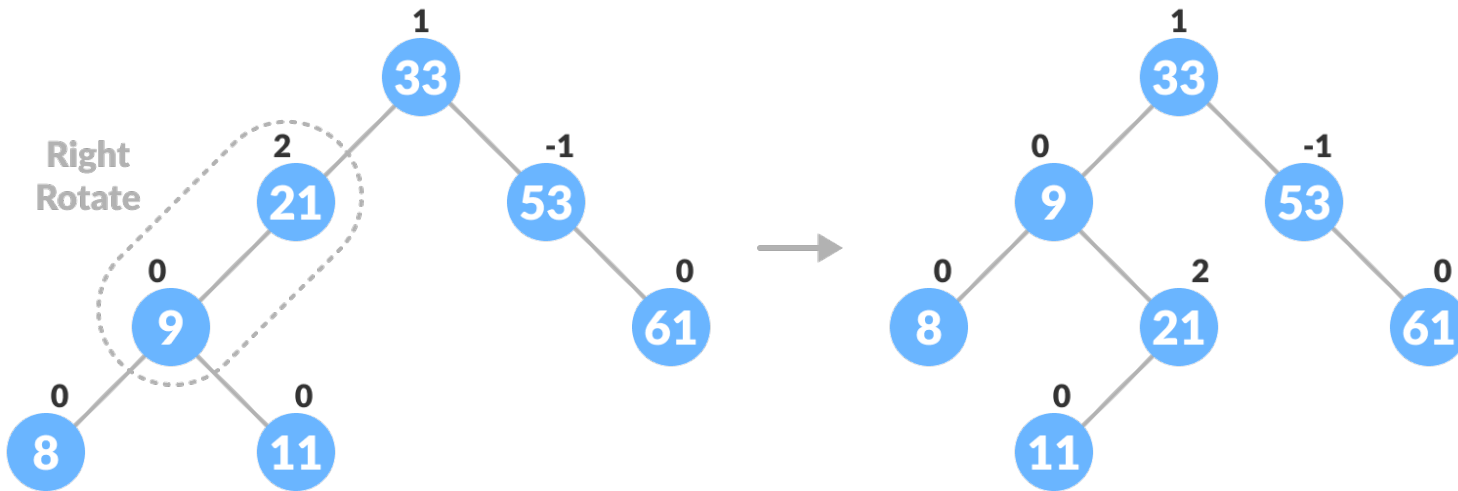
Calculate bF



Update bf

Deletion

- Rebalance the tree if the balance factor of any of the nodes is not equal to -1, 0 or 1.
 - If balanceFactor of currentNode > 1,
 - If balanceFactor of leftChild >= 0, do right rotation.
 - Else do left-right rotation.
 - If balanceFactor of currentNode < -1,
 - If balanceFactor of rightChild <= 0, do left rotation.
 - Else do right-left rotation.



Code

```
#include <iostream>
using namespace std;
```

```
class Node {
    public:
    int key;
    Node *left;
    Node *right;
    int height;
};
```

```
int max(int a, int b);
```

```
// Calculate height
int height(Node *N) {
    if (N == NULL)
        return 0;
    return N->height;
}
```

```
int max(int a, int b) {
    return (a > b) ? a : b;
}
```

```
// New node creation
Node *newNode(int key) {
    Node *node = new Node();
    node->key = key;
    node->left = NULL;
    node->right = NULL;
    node->height = 1;
    return node;
}
```

```
// Rotate right
Node *rightRotate(Node *y) {
    Node *x = y->left;
    Node *T2 = x->right;
    x->right = y;
    y->left = T2;
    y->height = max(height(y->left),
                    height(y->right)) +
                1;
    x->height = max(height(x->left),
                    height(x->right)) +
                1;
    return x;
}
```

// Rotate left

```
Node *leftRotate(Node *x) {  
    Node *y = x->right;  
    Node *T2 = y->left;  
    y->left = x;  
    x->right = T2;  
    x->height = max(height(x->left),  
                    height(x->right)) +  
                    1;  
    y->height = max(height(y->left),  
                    height(y->right)) +  
                    1;  
    return y;  
}
```

// Get the balance factor of each node

```
int getBalanceFactor(Node *N) {  
    if (N == NULL)  
        return 0;  
    return height(N->left) -  
           height(N->right);  
}
```

// Insert a node

```
Node *insertNode(Node *node, int key) {  
    // Find the correct postion and insert the node  
    if (node == NULL)  
        return (newNode(key));  
    if (key < node->key)  
        node->left = insertNode(node->left, key);  
    else if (key > node->key)  
        node->right = insertNode(node->right, key);  
    else  
        return node;
```

// Update the balance factor of each node and
// balance the tree

```
node->height = 1 + max(height(node->left),  
                       height(node->right));  
int balanceFactor = getBalanceFactor(node);  
if (balanceFactor > 1) {  
    if (key < node->left->key) {  
        return rightRotate(node);  
    } else if (key > node->left->key) {  
        node->left = leftRotate(node->left);  
        return rightRotate(node);  
    }  
}
```

```
}  
if (balanceFactor < -1) {  
    if (key > node->right->key) {  
        return leftRotate(node);  
    } else if (key < node->right->key) {  
        node->right = rightRotate(node->right);  
        return leftRotate(node);  
    }  
}  
return node;  
}
```

// Node with minimum value

```
Node *nodeWithMimumValue(Node *node) {  
    Node *current = node;  
    while (current->left != NULL)  
        current = current->left;  
    return current;  
}
```

// Delete a node

```
Node *deleteNode(Node *root, int key) {  
    // Find the node and delete it  
    if (root == NULL)  
        return root;  
    if (key < root->key)  
        root->left = deleteNode(root->left, key);  
    else if (key > root->key)  
        root->right = deleteNode(root->right, key);  
    else {
```

```
        if ((root->left == NULL) || (root->right == NULL)) {  
            Node *temp = root->left ? root->left : root->right;  
            if (temp == NULL) {  
                temp = root;  
                root = NULL;  
            } else  
                *root = *temp;  
            free(temp);  
        } else {  
            Node *temp = nodeWithMimumValue(root->right);  
            root->key = temp->key;  
            root->right = deleteNode(root->right, temp->key);  
        }  
    }  
}
```

```
if (root == NULL)  
    return root;
```

```

// Update the balance factor of each node and
// balance the tree
root->height = 1 + max(height(root->left), height(root->right));
int balanceFactor = getBalanceFactor(root);
if (balanceFactor > 1) {
    if (getBalanceFactor(root->left) >= 0) {
        return rightRotate(root);
    } else {
        root->left = leftRotate(root->left);
        return rightRotate(root);
    }
}
if (balanceFactor < -1) {
    if (getBalanceFactor(root->right) <= 0) {
        return leftRotate(root);
    } else {
        root->right = rightRotate(root->right);
        return leftRotate(root);
    }
}
return root;
}

```

```

// Print the tree
void printTree(Node *root) {
    cout << root->key << endl;
    printTree(root->left);
    printTree(root->right);
}
}

int main() {
    Node *root = NULL;
    root = insertNode(root, 33);
    root = insertNode(root, 13);
    root = insertNode(root, 53);
    root = insertNode(root, 9);
    root = insertNode(root, 21);
    root = insertNode(root, 61);
    root = insertNode(root, 8);
    root = insertNode(root, 11);
    printTree(root);
    root = deleteNode(root, 13);
    cout << "After deleting " << endl;
    printTree(root);
}

```